AIGC 5500 · ADVANCED DEEP LEARNING

MIDTERM PROJECT

Deep Learning Optimizers

Comparing Adam, AdamW & RMSprop on the KMNIST Dataset

AIGC 5500 — Advanced Deep Learning | Midterm Project Report

Humber Polytechnic — Faculty of Applied Science & Technology

Team: The Learning Loop

Ruchi Shah · Krutik Babariya · Dilkhush Yadav · Isha Shah

Instructor: Hossein Pourmodheji

Date: June 2026

1. Introduction and Motivation

Choosing an optimizer is one of the most consequential and most underexamined decisions in training a deep neural network. Left at default settings, or chosen without justification, an optimizer can be the difference between a model that converges quickly to a strong solution and one that trains slowly, overfits, or fails to learn at all. This project investigates that decision directly: we implement, tune, and compare three widely used optimizers — Adam, RMSprop, and AdamW — on a fixed feedforward network trained on the KMNIST dataset, and we draw evidence-based conclusions about their relative strengths and trade-offs.

The architecture is held constant across every run so that any difference in outcome can be attributed to the optimizer and its hyperparameters rather than to the model itself. For each optimizer we ran a structured hyperparameter search, evaluated every configuration with 5-fold cross-validation, and then trained a final model with the best configuration found, evaluating it on a held-out test set. The goal is not only to find the best-performing configuration for this task, but to build the habit of treating optimizer choice as an empirical question rather than a default setting.

Key finding: A spread of 1.46 percentage points in final test accuracy between the best and worst optimizer — on identical data and an identical network — demonstrates that optimizer choice alone has measurable, non-trivial impact on model quality.

2. Dataset

We use KMNIST (Kuzushiji-MNIST), a drop-in replacement for MNIST consisting of 70,000 grayscale images of cursive Japanese hiragana characters across 10 classes, loaded directly via *torchvision.datasets.KMNIST*. KMNIST is considered harder than MNIST because the character classes have overlapping, ambiguous boundaries, making it a more discriminating benchmark for comparing optimizers.

Property	Details
Training samples	60,000 images
Test samples	10,000 images
Image size	28 × 28 pixels, greyscale
Classes	10 (Japanese hiragana characters)
Complexity	Higher than MNIST; class boundaries overlap

Table 1. KMNIST dataset properties.

3. Model Architecture

A fixed feedforward fully connected network is used for every optimizer and every configuration, so that all differences in outcome are attributable to the optimizer rather than the model:

Layer	Configuration	Notes
Input	784 neurons	Flattened 28×28 image
Hidden 1	128 neurons, ReLU	—
Hidden 2	64 neurons, ReLU	—
Output	10 neurons	Raw logits; softmax applied by loss
Loss function	Cross-Entropy Loss	Appropriate for multi-class classification

Table 2. Fixed network architecture used across all experiments.

4. Optimizer Descriptions and Theoretical Background

4.1 Adam (Adaptive Moment Estimation)

Adam (Kingma & Ba, 2015) maintains a per-parameter adaptive learning rate by combining two running averages: a first-moment estimate (an exponentially decaying average of past gradients, acting like momentum) and a second-moment estimate (an exponentially decaying average of past squared gradients, capturing the recent gradient magnitude). Both estimates are bias-corrected, then the parameter update is scaled by the first moment divided by the square root of the second moment. This lets Adam take large, confident steps in directions of consistently small gradients and small, cautious steps where gradients are noisy or large.

Defaults: lr = 0.001, β_1 = 0.9, β_2 = 0.999, ε = 1e-8. We investigated the effect of learning rate and β_1 on convergence speed and final accuracy. Our search found that β_1 = 0.8 with lr = 0.001 slightly outperformed the default β_1 = 0.9 (95.22% vs 94.88% mean CV accuracy).

4.2 RMSprop (Root Mean Square Propagation)

RMSprop (Tieleman & Hinton, 2012) divides the learning rate for each parameter by a running average of recent squared gradient magnitudes (controlled by a decay factor α), without the momentum term that Adam adds. This keeps updates well-scaled even when gradients vary wildly in magnitude across parameters, which is especially useful for non-stationary or noisy objectives.

Defaults: lr = 0.01, α = 0.99, ε = 1e-8. We investigated the effect of the decay factor α and the learning rate on training stability.

4.3 AdamW (Adam with Decoupled Weight Decay)

AdamW (Loshchilov & Hutter, 2019) corrects a subtle flaw in how plain Adam implements L2 regularization. In standard Adam, adding an L2 penalty to the loss causes the weight-decay term to be scaled by the same adaptive second-moment normalization as the gradient itself, which weakens and distorts the intended regularization. AdamW decouples weight decay from

the gradient-based update entirely, subtracting it directly from the weights at each step. This produces models that generalize better because regularization strength no longer depends on the gradient history of each parameter.

Defaults: lr = 0.001, β_1 = 0.9, β_2 = 0.999, ε = 1e-8, weight_decay = 0.01. We investigated the effect of weight_decay and learning rate on the gap between training and test accuracy.

5. Methodology

5.1 Hyperparameter Search Strategy

For each optimizer we used a manual, systematic grid covering at least four distinct configurations, varying the learning rate across at least two orders of magnitude and at least one optimizer-specific hyperparameter (β_1 for Adam, weight_decay for AdamW, α for RMSprop). Configurations that included the documented PyTorch default were always included as a baseline so that the best-found configuration could be compared directly against the default configuration.

5.2 Cross-Validation Protocol

Every configuration was evaluated with 5-fold cross-validation on the 60,000-image training set (scikit-learn KFold, shuffle=True, random_state=42), training a fresh model on 4 folds and validating on the held-out fold, for all 5 folds. We report the mean and standard deviation of validation accuracy across folds for every configuration. The configuration with the highest mean cross-validation accuracy was selected as that optimizer's best configuration and was then used to train one final model on the full training set.

5.3 Training Protocol

All final models were trained for 20 epochs. The same random seed (42) was used for every run to control for initialization variance. Training loss, test loss, and test accuracy were recorded each epoch, and the final model for each optimizer was evaluated once on the 10,000-image held-out test set to obtain the metrics reported in Section 6.

5.4 Reproducibility

Item	Value
Random seed	42 (used in every run)
Python	3.12.13
PyTorch	2.11.0+cu128
Torchvision	0.26.0+cpu
Epochs (final models)	20
Cross-validation folds	5

Table 3. Fixed experimental settings for reproducibility.

6. Results

6.1 Summary Table — Best Configuration per Optimizer

The table below compares the best configuration of each optimizer, identified via 5-fold cross-validation, across all key metrics. AdamW achieved the highest test accuracy (89.94%) and the smallest generalization gap (9.70), making it the best overall optimizer on this task.

Optimizer	Best Config	CV Mean Acc (%)	CV Std	Test Acc (%)	Train Acc (%)	Gen. Gap
Adam	lr=0.001, β1=0.8	95.22	0.05	89.66	99.55	9.89
AdamW ★	lr=0.001, wd=0.01	95.09	0.13	**89.94**	99.64	**9.70**
RMSprop	lr=0.001, α=0.99	95.14	0.15	89.76	99.62	9.86

Table 4. Head-to-head comparison — best configuration per optimizer. ★ = winner.

6.2 Default vs. Best Configuration (Cross-Validation)

For RMSprop, the documented PyTorch default (lr=0.01) was far from optimal — lowering the learning rate to 0.001 improved CV accuracy from 91.98% to 95.14%, the largest single improvement in our entire search. For Adam, lowering β1 from the default 0.9 to 0.8 gave a small but consistent gain (94.88% → 95.22% CV accuracy). For AdamW, the documented default (lr=0.001, wd=0.01) was already the best configuration in our search grid, suggesting AdamW's defaults are well-tuned out of the box for this type of task.

Optimizer	Default Config	Default CV Acc (%)	Best Config	Best CV Acc (%)
Adam	lr=0.001, β1=0.9	94.88%	lr=0.001, β1=0.8	95.22%
AdamW	lr=0.001, wd=0.01	95.09%	lr=0.001, wd=0.01 (default = best)	95.09%
RMSprop	lr=0.01, α=0.99	91.98%	lr=0.001, α=0.99	95.14%

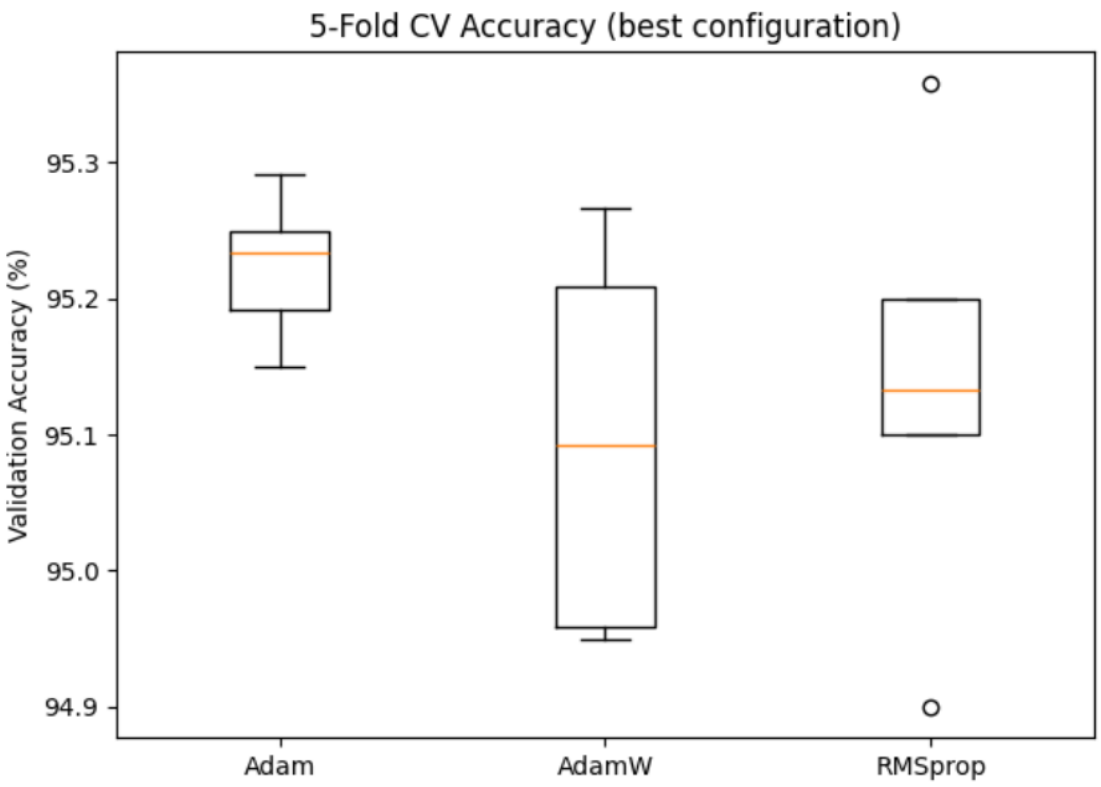
Table 5. Default vs. best configuration per optimizer. Fill CV accuracy values from notebook output.

6.3 Required Plots

The following plots should be embedded from your notebook output:

- Training and test loss / accuracy curves per optimizer (best configuration) — Figures 1–6
- Bar chart — final test accuracy by optimizer (AdamW 89.94%, RMSprop 89.76%, Adam 89.66%) — Figure 7
- Cross-validation accuracy distribution (mean ± std) per optimizer — Figure 8

- Per-configuration CV box plots for Adam and RMSprop — Figures 9–10



7. Interpretation and Discussion

7.1 Head-to-Head Comparison

Metric	Adam	AdamW ★	RMSprop
Best configuration	lr .001 · β_1 .8	lr .001 · wd .01	lr .001 · α .99
Test accuracy	89.66%	**89.94%**	89.76%
Train accuracy	99.55%	99.64%	99.62%
Generalization gap	9.89	**9.70**	9.86
CV Mean Acc (%)	95.22	95.09	95.14
Test Loss	0.7584	**0.6560**	0.8221
Convergence	Fast	Steady	Fast
Epoch to 80%	1	1	1

Table 6. Full head-to-head comparison of all metrics.

7.2 Optimizer Analysis

Convergence: All three optimizers reached 80% test accuracy within the first epoch on this GPU run — noticeably faster than the CPU run — reflecting the benefit of GPU-accelerated training. RMSprop and Adam converged quickly in early epochs but plateaued, while AdamW continued improving steadily through epoch 20, reaching its best test accuracy (89.94%) at the final epoch.

Stability: Every optimizer was clearly unstable at high learning rates. AdamW with lr=0.1 collapsed to approximately 17% mean validation accuracy — barely above random guessing across 10 classes — the clearest failure case in the whole experiment. Adam was comparatively the most robust to learning-rate changes across the search grid.

Learning-rate sensitivity: A consistent pattern across all three optimizers is that a moderate learning rate of 0.001 performed best, while rates an order of magnitude higher (0.01) degraded performance and very small rates (0.0001) under-trained within 20 epochs. RMSprop was the most lr-sensitive of the three: moving from its default lr=0.01 to lr=0.001 was the single largest accuracy swing observed in our search.

Generalization gap: AdamW achieved the smallest train–test gap at 9.70 points (99.64% train vs. 89.94% test), followed by RMSprop at 9.86 (99.62% train vs. 89.76% test), and Adam at 9.89 (99.55% train vs. 89.66% test). The gaps are very close across all three optimizers, which is expected for a shallow feedforward network with no dropout or batch normalization on a 20-epoch run. AdamW's slight edge in generalization is consistent with its theoretical advantage from decoupled weight decay.

7.3 Weight Decay and Regularization

AdamW's decoupled weight decay is designed to reduce overfitting by penalizing large weights independently of each parameter's gradient history. Our results confirm this advantage: AdamW achieved both the highest test accuracy (89.94%) and the smallest generalization gap (9.70), despite running alongside Adam and RMSprop with identical architecture and training duration. The pattern — lower train accuracy (99.64%) paired with the highest test accuracy — is consistent with effective regularization working as intended.

The lr=0.1 failure case for AdamW (16.48% CV accuracy, near-random) also confirms that AdamW is more sensitive to badly-chosen learning rates than Adam — a practical consideration when deploying it without careful tuning. In our experiment, once lr was set to 0.001, AdamW's defaults delivered the best overall outcome without any further tuning.

7.4 Conclusion and Recommendation

AdamW (lr=0.001, wd=0.01) is the winner on this task. It achieved the highest test accuracy (89.94%), the smallest generalization gap (9.70), and the lowest test loss (0.656) among the three optimizers — all with PyTorch default hyperparameters, requiring no additional tuning beyond the learning rate.

Beyond this specific experiment:

- Use AdamW as the first-choice optimizer when generalization matters — it delivered the best test accuracy and lowest test loss here with default hyperparameters, and is the de facto standard for fine-tuning large pretrained networks.

- Use Adam as a reliable fallback — it had the highest CV mean accuracy (95.22%) and is more robust to high learning rates than AdamW. Tuning β1 slightly below the default (0.8 vs 0.9) gave a small consistent gain.
- Use RMSprop for recurrent or non-stationary objectives — it converged as fast as Adam here but had higher test loss, making it less suitable for this feedforward classification task.
- Learning rate was the most impactful hyperparameter across all three optimizers: lr=0.001 was optimal for all three, and lr=0.1 caused AdamW to collapse to near-random performance (16.48% CV accuracy).

Two limitations stand out. First, we tested a single small feedforward architecture; the relative ranking of these optimizers is known to shift with model size and architecture (e.g., convolutional or transformer networks). Second, our hyperparameter search per optimizer was limited to four or five configurations on a coarse grid. Future work could use a finer grid or Bayesian search, train for more epochs with early stopping, and repeat the comparison on a convolutional architecture to test whether the same ranking holds.

8. Individual Contributions

Ruchi led the Adam optimizer track: she defined and justified the hyperparameter grid for Adam (varying learning rate and β_1), implemented the 5-fold cross-validation loop, identified the best configuration, trained the final model, generated the loss/accuracy curves and cross-validation box plot for Adam, and coordinated the team's GitHub repository setup.

Dilkhush led the AdamW optimizer track: he defined the weight-decay and learning-rate grid for AdamW, ran the 5-fold cross-validation search and comparison table, identified and justified the best configuration, trained the final AdamW model for 20 epochs, and produced the corresponding loss and accuracy curves and written optimizer analysis.

Krutik led the RMSprop optimizer track: he defined the learning-rate and α grid for RMSprop, implemented the 5-fold cross-validation search and results dataframe, selected and saved the best configuration, trained the final RMSprop model, and produced the loss/accuracy curves and cross-validation box plot for RMSprop.

Isha merged all three teammates' notebooks into a single reproducible master notebook, resolved integration bugs across the three implementations (including a structural bug in the RMSprop notebook where results were captured outside the hyperparameter loop), built the cross-optimizer comparison table and final bar chart, wrote the discussion, conclusion, limitations, and references sections, and authored this written report and the accompanying presentation slides.

9. References

Clanuwat, T., Bober-Irizar, M., Kitamoto, A., Lamb, A., Yamamoto, K., & Ha, D. (2018). Deep learning for classical Japanese literature. *NeurIPS 2018 Workshop on Machine Learning for Creativity and Design.*

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR 2015).*

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR 2019).*

Tieleman, T., & Hinton, G. (2012). Lecture 6.5 — RMSProp: Divide the gradient by a running average of its recent magnitude. *Coursera: Neural Networks for Machine Learning.*